

# Programmierung 1

im Wintersemester 2025/26

## Praktikumsaufgabe 10

### Zahlen und Statisches (Kap. 10)

Review: 20. Januar 2026 (DFIW) und 21. Januar 2026 (PI)

In dieser Praktikumsaufgabe arbeiten Sie mit statischen Methoden und Konstanten sowie mit Zahlen.

In **Aufgabe 1** implementieren und verwenden Sie Konstanten und statische Hilfsmethoden und formatieren Zahlen für die Ausgabe.

In **Aufgabe 2** implementieren Sie einen Algorithmus zur Ermittlung von Primzahlen.

In **Aufgabe 3** implementieren Sie Code, der eine Testspezifikation erfüllt.

### **Heads up!**

Für diese und die folgenden Praktikumsaufgaben gelten weiterhin die Grundsätze der professionellen Softwareentwicklung, die Sie bereits in den vorherigen Praktikumsaufgaben kennengelernt haben. *Dies wird auch in der Klausur von Ihnen erwartet.* Und nach wie vor gilt: Lösen Sie die Aufgaben nicht allein, sondern mindestens zu zweit! Zum Beispiel mit Ihrer Coding-Partner:in. Denn in der Gruppe lernt es sich besser.

**Wir als Ihre Lernbegleiter:innen unterstützen Sie gerne bei der Lösung der Aufgaben.**

# 1 Zahlen

## Lernziele

- ☐ Sie erstellen und verwenden Konstanten unter Verwendung geeigneter Modifier (Modifikatoren).
- ☐ Sie implementieren statische Hilfsmethoden, die nicht vom Wert einer bestimmten Instanzvariablen abhängen.
- ☐ Sie formatieren Zahlen für die Ausgabe, um sie lesbarer zu machen.

In dieser Aufgabe beschäftigen Sie sich mit Konstanten, statischen Hilfsmethoden und der Formatierung von Zahlen bei der Ausgabe.

## 1.1 Konstanten

Implementieren Sie eine Klasse `SpeedOfLight`, die die **Lichtgeschwindigkeit** als Konstante zur Verfügung stellt!

*Hinweis:* Die Lichtgeschwindigkeit ist 299 792 458 m/s.

Schreiben Sie eine korrekt benannte Testklasse, die die Lichtgeschwindigkeit auf der Konsole ausgibt.

## 1.2 Statische Hilfsmethoden

Erweitern Sie die Klasse um eine Methode **double fraction (double)**, die die Lichtgeschwindigkeit mit einem Faktor zwischen 0 und 1 (jeweils einschließlich) multipliziert!

Erweitern Sie die Testklasse so, dass die Methode mit verschiedenen Faktoren aufgerufen und das Ergebnis auf der Konsole ausgegeben wird.

## 1.3 Zahlenformatierung

Fügen Sie der Klasse eine Methode `void printFraction(double)` hinzu, die die obige Methode verwendet, das Ergebnis formatiert und es direkt auf der Konsole ausgibt!

*Wichtig:* Die Ausgabe soll Tausendertrennzeichen enthalten und auf zwei Dezimalstellen gerundet werden. Achten Sie bitte darauf, die deutsche Formatierung zu verwenden, d. h. Komma als Dezimaltrenner und Punkt als Tausendertrenner – und zwar unabhängig von den Systemeinstellungen Ihres Computers (Tipp: `Locale.GERMANY`).

Anstatt das Ergebnis der Berechnung selbst auszugeben, soll die Testklasse diese neue Methode verwenden.

## 1.4 Reflexion im Coding-Team

Diskutieren Sie im Coding-Team, um Ihr Verständnis zu vertiefen!

### 1.4.1 Konstanten und ihre Anwendung in der Praxis

**Warum sind Konstanten in der Softwareentwicklung unverzichtbar und wie tragen sie zur besseren Lesbarkeit, Wartbarkeit und Fehlervermeidung bei?**

- Welche Probleme können auftreten, wenn feste Werte (wie z. B. die Lichtgeschwindigkeit) nicht als Konstanten, sondern direkt im Code verwendet werden?
- Wie kann die Verwendung von Konstanten die Zusammenarbeit in einem Entwicklungsteam erleichtern?
- Welche Regeln oder bewährte Praktiken sollten bei der Deklaration und Benennung von Konstanten beachtet werden, damit sie im Code eindeutig und sinnvoll verwendet werden können?

### 1.4.2 Anwendung statischer Methoden

**Wann ist es sinnvoll, eine Methode als statische Methode zu implementieren und was sind die Vorteile gegenüber Instanzmethoden?**

- Was unterscheidet statische Methoden von Instanzmethoden und in welchen Szenarien sind statische Methoden besonders sinnvoll?
- Warum kann es vorteilhaft sein, Methoden wie `fraction` oder `printFraction` statisch zu definieren, anstatt sie einer Instanz zuzuordnen?
- Welche Nachteile könnte die übermäßige Verwendung von statischen Methoden haben und wie kann man diese vermeiden?

### 1.4.3 Gute Praxis bei der Formatierung von Zahlen

**Welchen Einfluss hat die Formatierung von Zahlen auf die Benutzerfreundlichkeit eines Programms und welche Prinzipien sollten beachtet werden, um eine klare und lesbare Darstellung zu gewährleisten?**

- Welche Herausforderungen könnten Benutzer:innen ohne gut formatierte Ausgaben haben (z. B. ohne Tausendertrennzeichen)?
- Wie sollte das Format einer Zahl in einem Programm an den Kontext angepasst werden, in dem sie angezeigt wird (z. B. wissenschaftliche Berechnungen vs. Unternehmenssoftware)?
- Welche Werkzeuge oder Bibliotheken können bei der Formatierung von Zahlen helfen und wie kann ihr Einsatz die Qualität des Codes verbessern?

## 2 Sieb des Eratosthenes

### Lernziele

- ☐ Sie implementieren einen Algorithmus zur Bestimmung von Primzahlen.
- ☐ Sie optimieren Ihre Implementierung, um die Laufzeit des Algorithmus zu verringern.

In dieser Aufgabe implementieren Sie den Algorithmus *Sieb des Eratosthenes* zur Bestimmung von Primzahlen.

Obwohl Sie bei der Entwicklung von Software in der Regel auf Bibliotheken zurückgreifen können, die solche Berechnungen und andere Aufgaben für Sie übernehmen, ist es dennoch wichtig, dass Sie in der Lage sind, derartige Algorithmen zu verstehen und zu implementieren. Dies schult nicht zuletzt Ihr algorithmisches Denken, das für die Softwareentwicklung unerlässlich ist.

**Algorithmisches Denken** ist eine grundlegende Fähigkeit um komplexe Probleme systematisch zu analysieren und effiziente Lösungen zu entwickeln:

**Förderung der Problemlösungskompetenz:** Das Verstehen und Implementieren von Algorithmen fördert die Fähigkeit, Probleme in kleinere, überschaubare Teile zu zerlegen und schrittweise Lösungen zu entwickeln.

**Verbesserung der Code-Optimierung:** Ein tieferes Verständnis von Algorithmen ermöglicht es Entwickler:innen, effizienteren und optimierten Code zu schreiben.

**Flexibilität und Anpassungsfähigkeit:** Die Fähigkeit, Algorithmen zu verstehen und zu implementieren, ermöglicht es Entwickler:innen, maßgeschneiderte Lösungen für spezifische Probleme zu entwickeln, die möglicherweise nicht von Standardbibliotheken abgedeckt werden.

**Grundlage für weiterführende Konzepte:** Algorithmisches Denken bildet die Grundlage für das Verständnis komplexerer informatischer Konzepte und Techniken.

**Interdisziplinäre Anwendbarkeit:** Die durch algorithmisches Denken erworbenen Fähigkeiten sind nicht nur in der Informatik, sondern auch in anderen Bereichen wie Mathematik, Naturwissenschaften und sogar im täglichen Leben anwendbar.

Die Stärkung des algorithmischen Denkens leistet also einen wesentlichen Beitrag zu Ihrer Ausbildung als kompetente Softwareentwickler:in.

**Heads up!** Verwenden Sie testgetriebene Entwicklung, um den Algorithmus schrittweise zu implementieren und zu optimieren.

## 2.1 Einfache Implementierung

Implementieren Sie das Sieb des Eratosthenes zur Bestimmung von Primzahlen!

Das Sieb des Eratosthenes ist ein Algorithmus zur Bestimmung einer Liste aller Primzahlen, die kleiner oder gleich einer gegebenen Zahl sind. Es ist nach dem griechischen Mathematiker Eratosthenes benannt. Die Grundidee besteht darin, Primzahlen systematisch zu identifizieren, indem aus einer Liste von Zahlen alle Vielfachen von Primzahlen entfernt werden.

*Hinweis:* Verwenden Sie zwei ineinander geschachtelte Schleifen. Die äußere Schleife durchläuft die zu untersuchenden Zahlen. Die innere Schleife markiert alle Vielfachen der aktuellen Zahl als nicht-prim.

*Wichtig:* Beginnen Sie mit einer möglichst einfachen und leicht verständlichen Implementierung. Optimierungen nehmen Sie im nächsten Teil der Aufgabe vor.

## 2.2 Optimierte Implementierung

Optimieren Sie Ihre Implementierung des Siebs des Eratosthenes!

Ziel der Optimierung ist es, die Laufzeit des Algorithmus zu verkürzen, insbesondere bei großen Zahlen. Finden Sie heraus, wie Sie Ihre Implementierung des Siebes entsprechend optimieren können. Implementieren Sie mindestens eine Optimierung.

*Tipp:* Überlegen Sie gemeinsam im Coding-Team, warum es ausreicht, nur bis zur Quadratwurzel der maximalen Zahl zu prüfen. Welche Zahlen sind bereits als Vielfache gekennzeichnet?

*Hinweis:* Die zuvor implementierten Tests dienen als Sicherheitsnetz, um die korrekte Funktionalität der Implementierung aufrechtzuerhalten.

## 2.3 Reflexion im Coding-Team

Diskutieren Sie im Coding-Team, um Ihr Verständnis zu vertiefen!

### 2.3.1 Der Algorithmus und seine Funktionsweise

**Wie funktioniert das Sieb des Eratosthenes, um Primzahlen zu finden, und wie werden die einzelnen Schritte des Algorithmus systematisch umgesetzt?**

- Wie wird die Zahl 1 in Ihrer Implementierung behandelt? Ist sie mathematisch gesehen eine Primzahl und spiegelt Ihr Code dies korrekt wider?
- Welche Grundidee steckt hinter der Eliminierung der Vielfachen im Sieb des Eratosthenes und warum bleibt nur eine Liste von Primzahlen übrig?
- Welche Rolle spielt die äußere Schleife und warum reicht es aus, nur bis zur Quadratwurzel der maximalen Zahl zu iterieren?
- Wie wird der Algorithmus im Code implementiert und welche Datenstrukturen (z. B. Arrays oder Listen) sind dafür am besten geeignet?

### 2.3.2 Möglichkeiten der Optimierung und Effizienzsteigerung

**Wie kann die Laufzeit des Siebs des Eratosthenes durch Optimierungen verbessert werden und welche Techniken eignen sich zur Effizienzsteigerung?**

- Was sind die Schwächen der naiven Implementierung des Siebes und welche Teile des Algorithmus verursachen die größte Laufzeit?

- Wie könnte die Verwendung eines **boolean**-Arrays oder das Überspringen gerader Zahlen (außer 2) die Effizienz des Algorithmus verbessern?
- Welche anderen Optimierungstechniken (z. B. Parallelverarbeitung oder Speicheroptimierung) könnten für große Zahlenbereiche nützlich sein?

### 2.3.3 Bedeutung von testgetriebener Entwicklung

**Wie unterstützt die testgetriebene Entwicklung die Implementierung und Optimierung des Siebs des Eratosthenes und welche Vorteile bietet sie im Entwicklungsprozess?**

- Wie helfen Tests, die Korrektheit des Algorithmus während der Entwicklung und Optimierung sicherzustellen?
- Warum ist es wichtig, den Algorithmus in kleinen, überprüfbaren Schritten zu implementieren, und wie unterstützen Tests diesen Ansatz?
- Wie können Fehler oder unerwartete Ergebnisse ohne Tests auftreten und wie bieten Tests ein Sicherheitsnetz bei Änderungen am Code?



## 3 Implementierung gegen Testspezifikation

### Lernziele

- ☐ Sie implementieren Code, der eine gegebene Spezifikation in Form von Tests erfüllt.
- ☐ Sie verstehen bestehenden Code, so dass Sie ihn verwenden können.
- ☐ Sie verwenden Methoden der Klasse `Math`.
- ☐ Sie verwenden **Autoboxing und Unboxing**.

In dieser Aufgabe implementieren Sie Code, der eine vorgegebene Spezifikation erfüllt. Die Spezifikation wird in Form von Tests vorgegeben.

Als Softwareentwickler:in ist es eine Ihrer Hauptaufgaben, Code zu entwickeln, der eine gegebene Anforderung bzw. Spezifikation erfüllt. Im Idealfall liegt eine solche Spezifikation in Form von ausführbaren Tests vor. Diese Tests dienen auch als konkretes Beispiel, anhand dessen Sie nicht nur Ihre Implementierung überprüfen, sondern auch Missverständnisse mit anderen Entwickler:innen, Manager:innen, Tester:innen und anderen Beteiligten vermeiden können. Eine solche Spezifikation wird auch *specification by example* oder **executable test-based specification** genannt.<sup>1</sup>

**Heads Up!** Wenn ein Test fehlschlägt, analysieren Sie die erwartete und die tatsächliche Ausgabe. Überlegen Sie, welche Bedingungen im Code nicht erfüllt wurden.

### 3.1 Executable Test-Based Specification

Kopieren Sie den gegebenen Test in Ihren Editor und lassen Sie ihn kompilieren und ausführen!

Der gegebene Test prüft, ob die beiden Methoden `min` und `max` der Klasse `ListMinMax` korrekt implementiert sind.<sup>2</sup>

```
1 import java.util.ArrayList;  
2 import java.util.List;
```

<sup>1</sup>Sie haben hoffentlich die große Parallele zur *testgetriebenen Entwicklung* bemerkt.

<sup>2</sup>Ein guter Test prüft sowohl Grenzfälle (z. B. `null` oder leere Listen) als auch typische Eingabewerte. Warum ist es wichtig, solche Fälle zu testen?

```
3
4 public class ListMinMaxTestDrive {
5     public static void main(String[] args) {
6         final ArrayList<Integer> nullList = null;
7
8         final ArrayList<Integer> emptyList = new ArrayList<>(List.of());
9
10        final ArrayList<Integer> singleElement = new
11            ↪ ArrayList<>(List.of(1058));
12
13        final ArrayList<Integer> twoElements = new ArrayList<>(List.of(-11,
14            ↪ 74));
15
16        final ArrayList<Integer> list = new ArrayList<>(List.of(-91, -64, 20,
17            ↪ 90, -54, -95, -81, 0, 79));
18
19        assertEquals(Integer.MAX_VALUE, ListMinMax.min(nullList));
20        assertEquals(Integer.MAX_VALUE, ListMinMax.min(emptyList));
21        assertEquals(1058, ListMinMax.min(singleElement));
22        assertEquals(-11, ListMinMax.min(twoElements));
23        assertEquals(-95, ListMinMax.min(list));
24
25        assertEquals(Integer.MIN_VALUE, ListMinMax.max(nullList));
26        assertEquals(Integer.MIN_VALUE, ListMinMax.max(emptyList));
27        assertEquals(1058, ListMinMax.max(singleElement));
28        assertEquals(74, ListMinMax.max(twoElements));
29        assertEquals(90, ListMinMax.max(list));
30    }
31
32    public static void assertEquals(int expected, int actual) {
33        if (expected == actual) {
34            System.out.println("[PASS] Expected: " + expected + " - Actual: " +
35                ↪ actual);
36        } else {
37            System.out.println("[FAIL] Expected: " + expected + " - Actual: " +
38                ↪ actual);
39        }
40    }
41}
```

**Hinweis:** Beim Kopieren von Code aus PDF-Dateien können Sonderzeichen (wie < > oder das Minuszeichen) mitunter fehlerhaft übertragen werden. Sollte der Code nicht kompilieren, prüfen Sie bitte beispielsweise die Generics (wie `ArrayList<Integer>`) und die negativen Zahlen.

Implementieren Sie die Klasse `ListMinMax` so, dass der Test kompiliert und ausgeführt werden kann. Schreiben Sie dazu leere Implementierungen der Methoden `min` und `max`.<sup>3</sup>

Bei der Ausführung soll die folgende Ausgabe erscheinen:

```
[FAIL] Expected: 2147483647 - Actual: 0
[FAIL] Expected: 2147483647 - Actual: 0
[FAIL] Expected: 1058 - Actual: 0
[FAIL] Expected: -11 - Actual: 0
[FAIL] Expected: -95 - Actual: 0
[FAIL] Expected: -2147483648 - Actual: 0
[FAIL] Expected: -2147483648 - Actual: 0
[FAIL] Expected: 1058 - Actual: 0
[FAIL] Expected: 74 - Actual: 0
[FAIL] Expected: 90 - Actual: 0
```

### 3.2 `ListMinMax.min()`

Implementieren Sie die Methode `min` der Klasse `ListMinMax` so, dass Ihre Implementierung den Test besteht!

*Wichtig:* Verwenden Sie die Methode `Math.min(int a, int b)`!

### 3.3 `ListMinMax.max()`

Implementieren Sie die Methode `max` der Klasse `ListMinMax` so, dass Ihre Implementierung den Test besteht!

*Wichtig:* Verwenden Sie die Methode `Math.max(int a, int b)`!

---

<sup>3</sup>Siehe Aufgabe 1.4 Testgetriebene Implementierung in Praktikumsaufgabe 5.

## 3.4 Reflexion im Coding-Team

Diskutieren Sie im Coding-Team, um Ihr Verständnis zu vertiefen!

### 3.4.1 Die Rolle von Testspezifikationen

**Wie können Testspezifikationen helfen, klare Anforderungen zu definieren, Missverständnisse zu vermeiden und die Zusammenarbeit im Entwicklungsteam zu verbessern?**

- Warum wird bei einer leeren Liste der Wert `Integer.MAX_VALUE` als Minimum erwartet? Diskutieren Sie die Vor- und Nachteile dieses Rückgabewerts im Vergleich zum Werfen einer Ausnahme.
- Was sind die Vorteile einer Testspezifikation gegenüber einer rein schriftlichen Anforderungsbeschreibung?
- Wie können Testspezifikationen Missverständnisse zwischen Entwickler:innen, Tester:innen und anderen Stakeholdern vermeiden?
- Welche Herausforderungen können bei der Implementierung gegen eine Testspezifikation auftreten und wie können diese gelöst werden?

### 3.4.2 Testmethoden und Fehlersuche

**Wie können die Ergebnisse der Tests genutzt werden, um Fehler in der Umsetzung systematisch zu erkennen und zu beheben?**

- Was ist zu tun, wenn ein Test fehlschlägt? Welche Schritte helfen, die Fehlerursache zu finden?
- Welche Informationen liefern Testausgaben und wie können diese genutzt werden, um gezielte Änderungen am Code vorzunehmen?

- Warum ist es wichtig, mit kleinen, schrittweisen Änderungen zu arbeiten, anstatt große Anpassungen vorzunehmen, um Tests zum Bestehen zu bringen?

### 3.4.3 Math-Methoden sowie Autoboxing und Unboxing

**Wie tragen Java-Features wie Autoboxing und Unboxing sowie Methoden der Klasse `Math` dazu bei, die Effizienz und Lesbarkeit von Code zu verbessern?**

- Was sind die Vorteile von Autoboxing und Unboxing im Vergleich zur manuellen Konvertierung zwischen primitiven Typen und Objekten?
- Wie erleichtern Methoden wie `Math.min()` und `Math.max()` die Implementierung, und welche Alternativen wären ohne diese Methoden notwendig?
- Welche möglichen Probleme oder Fallstricke gibt es bei der Verwendung dieser Java-Features und wie können diese vermieden werden?